VeriHarness: What Should a Validator Tell an Agent?

Anonymous Author(s)

ABSTRACT

We study iterative software systems in which a *model* (an LLM) generates a candidate and a *validator* (external checking code) returns pass or a failure. We call the repeated model–validator–feedback sequence an *agent loop*. What should the validator return to the next call? VeriHarness is a code-as-harness prototype in which LLMs generate every leaf action while code controls validation, acceptance, budgets, and traces. We compare raw diagnostics with feedback that exposes a failure location, expected alternatives, and the observed value.

Our primary experiment contains 400 oracle-blind rows: four repair policies, 50 paired TextWorld games, two model families, and a four-call cap. Structured verifier feedback raises success from 14/50 to 36/50 for Qwen2.5-Coder-14B (+44 points, 95% paired-bootstrap interval 28–60) and from 8/50 to 29/50 for Llama-3.1-8B (+42 points, 28–56). However, rendering the same information as untyped prose yields 35/50 and 29/50. Typed structure itself changes success by only +2 and 0 points. Removing expected alternatives largely removes the gain. The ordering persists across call budgets and sampled decoding, but not on a small public-test HumanEval scope check. Across 880 rows and 2,652 LLM calls, the evidence supports a narrow principle: expose explicit repair fields; do not attribute the benefit to JSON typing alone.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Intelligent agents.*

KEYWORDS

LLM agents, verifier feedback, repair, agent harnesses, TextWorld

ACM Reference Format:

Anonymous Author(s). 2026. VeriHarness: What Should a Validator Tell an Agent?. In *International Workshop on Agentic AI for Next-Generation Software Development (AgenticDev '26), October 12, 2026, Munich, Germany.* ACM, New York, NY, USA, 4 pages.

1 INTRODUCTION

We use three terms throughout. A *model* is the LLM called to generate one candidate action or artifact. A *validator* is external code that checks a candidate and returns either pass or a failure description. An *agent loop* is the bounded sequence model → validator → feedback, repeated until validation passes or the call budget is exhausted. In software development, such a loop may write a patch or plan, invoke a test or execution environment, and react to failure. ReAct, Reflexion, Self-Refine, and self-debugging establish the value of iterative feedback [1, 6, 7, 9]. Coding agents also benefit from purpose-built interfaces [8]. Yet one interface remains underspecified: *what exactly should a validator tell the next model call?*

Suppose command 1 in a plan is invalid. A raw retry sees "Command 1 is not admissible." A repair interface can additionally expose the observed command and the commands admissible at that state. This may help because the information is actionable, because the locus is explicit, or because a typed record is easier to parse. Prior evidence did not separate these explanations.

We use VeriHarness, a code-as-harness prototype, to ask:

RQ1. Does structured verifier feedback with expected alternatives improve repair over raw diagnostics under equal compute?

RQ2. Is any gain caused by typed syntax, structured content, or both?

Replications across models, call budgets, and decoding, plus a public-test code scope check, test the robustness and boundary of RQ1–RQ2; they are not a third research question.

Our answer is deliberately narrow. Structured verifier feedback produces a large, replicated gain on verifier-rich TextWorld plans. Typed JSON and information-matched prose have indistinguishable success. A location without expected alternatives is insufficient. The main contribution is therefore an empirical decomposition of validator feedback, plus a reproducible harness for testing it.

2 BACKGROUND AND SYSTEM

Reflexion stores verbal feedback, while Self-Refine lets an LLM produce and use its own critique [6, 7]. Execution-guided methods reuse failed programs and test messages [1]. SWE-bench and SWE-agent show the importance of executable evaluation and agent–computer interfaces [5, 8]. VeriHarness studies a complementary boundary: deterministic validator output passed to the next probabilistic leaf.

2.1 Code as the Control Plane

VeriHarness separates generation from acceptance (Fig. 1). A principal orchestrator builds a compact state pack and dispatches bounded LLM workers. Each worker returns a structured candidate. External gates check schema, artifacts, and environment behavior. Only gates accept a candidate; a worker's self-assessment cannot do so. Every prompt, raw response, parsed output, gate result, model setting, and elapsed time is persisted before the next call.

Code is preferable here to prose as the *control plane*. A prompt-only loop asks the model to remember workflow state, count calls, interpret its own completion, and carry an expanding history in context. Those duties compete with candidate generation, and model-declared completion cannot independently validate the model's output. Code instead keeps durable state outside the prompt, enforces budgets and acceptance gates deterministically, and permits the same tasks and failures to be replayed across policies. The model receives only the compact task state and current repair feedback. This is an engineering rationale for VeriHarness's architecture; the experiment compares feedback policies, not code control against prompt-only control.

The failure encoder maps gate-specific details to a common interface. For an invalid TextWorld action it can expose a stable label, the command index, the admissible actions in environment order,

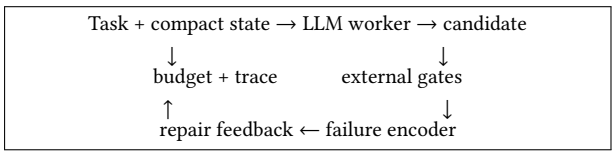

Figure 1: LLMs generate leaf candidates; code controls state, validation, acceptance, budgets, and traces.

Table 1: Feedback controls used in the primary experiment.

Policy	Feedback after failure
RawDiag	Raw validation message.
SameNL	Exact location, expected alternatives, and observed value in ordinary prose, without typed labels or keys.
LocObs	Typed label, location, and observed value; no expected alternatives.
TypedFields	Typed label, location, expected alternatives, and observed value.

and the rejected action. Oracle-blind evaluation excludes hidden post-hoc outcomes from repair feedback.

2.2 Feedback Policies

All policies receive the same task state, rejected answer, gate, model, and call cap. They differ only after a failed candidate (Table 1).

SameNL and TypedFields contain the same actionable field values; their contrast isolates representation. LocObs and TypedFields share typed structure; their contrast isolates expected alternatives. RawDiag versus SameNL measures the value of adding structured semantic content to the raw message. Here, *structured* means explicit repair components—location, expected alternatives, and observation—whether rendered as prose or JSON.

3 STUDY DESIGN

Primary tasks. We generate 50 fixed TextWorld games [2], with seeds 20261051–20261100. Each has three rooms, four objects, and a two-step quest. A candidate is a JSON plan of at most four commands, executed from fresh state. Success requires a terminal win. Invalid actions return their index and up to the first 12 admissible commands in the environment's deterministic order; we neither rank nor randomize this list. The primary matrix is 50 games × four policies × two models, or 400 rows.

Models and decoding. We use Qwen2.5-Coder-14B-Instruct-AWQ [4] on an NVIDIA L4 and Meta-Llama-3.1-8B-Instruct-AWQ-INT4 [3] on a T4, served by vLLM. Primary decoding is greedy with 512 output tokens and a four-call cap. Model conditions are never mixed across serving backends.

Robustness lanes. First, on the same 15-game subset and both models, we compare RawDiag and TypedFields at budgets 2, 4, 6, and 8. Budget-4 rows are reused from the primary matrix. Second, Qwen runs 20 games at temperature 0.3, top-p 0.9, and three inference seeds; this lane compares RawDiag, SameNL, and TypedFields. Third, a 15-task HumanEval scope check exposes one deterministically selected public assertion to the repair gate while retaining the complete official tests for post-hoc success. These lanes contribute 480 additional rows, for 880 total and 2,652 realized LLM calls.

Table 2: Primary TextWorld results: 50 paired games per model, budget 4.

	Qwen-14B/L4		Llama-8B/T4	
Policy	Solved	Calls	Solved	Calls
RawDiag	14 (28%)	164	8 (16%)	179
LocObs	18 (36%)	155	9 (18%)	174
SameNL	35 (70%)	147	29 (58%)	161
TypedFields	**36 (72%)**	**130**	**29 (58%)**	**149**

Analysis. Games are paired by seed. Primary success differences use 10,000 paired bootstrap samples and two-sided exact McNemar tests. We apply Holm correction to four planned contrasts within each model. Sampled-decoding intervals resample 20 game clusters, retaining all three repeats; p-values use 100,000 game-level sign flips and Holm correction. Budget and HumanEval lanes are small robustness checks, so we emphasize counts and intervals rather than dichotomizing their p-values. Saved-transcript whitespace counts are a prompt size proxy, not tokenizer or billing tokens.

4 RESULTS

4.1 Structured Verifier Feedback Produces the Gain

Table 2 reports terminal success. TypedFields improves over RawDiag by 44 points for Qwen (95% interval 28–60; Holm-adjusted exact $p = 3.15\times10^{-5}$) and 42 points for Llama (28–56; $p = 3.81\times10^{-6}$). The result therefore replicates across a coder model and a smaller general model.

The ablations identify the active information (Table 3). SameNL beats RawDiag by 42 points on both models, with no baseline-only successes. In contrast, TypedFields versus the information-matched SameNL is only +2 points for Qwen and 0 for Llama; both intervals include zero and both adjusted p-values are 1. Typed structure does not explain success.

Expected alternatives do. LocObs, which retains typed label, location, and observation but omits alternatives, is near RawDiag. Adding alternatives raises success by 36 points for Qwen and 40 for Llama. Final invalid-action episodes fall from 35 to 2 (raw to same-information prose) for Qwen and from 41 to 4 for Llama. A location says where repair is needed; admissible alternatives say what repair is possible.

Typing has a smaller efficiency effect. At equal success, TypedFields uses 17 fewer calls than SameNL for Qwen (paired mean −0.34 calls/game, interval −0.60 to −0.06) and 12 fewer for Llama (−0.24, −0.44 to −0.04). Average retry prompts are similar: 701 versus 716 words for Qwen and 706 versus 722 for Llama. Total realized prompt words are 14% and 10% lower for typed feedback

Table 3: Paired contrasts for Qwen2.5-Coder-14B-Instruct-AWQ and Meta-Llama-3.1-8B-Instruct-AWQ-INT4. CIs are paired bootstrap; p_H is Holm-adjusted exact McNemar within model.

Model	Treatment vs. baseline	Δ	95% CI	p_H
Qwen	SameNL vs. raw	+42	[28,56]	3.8×10^{-6}
Qwen	Typed vs. SameNL	+2	[−8,12]	1.0
Qwen	Typed vs. LocObs	+36	[20,52]	2.4×10^{-4}
Llama	SameNL vs. raw	+42	[28,56]	3.8×10^{-6}
Llama	Typed vs. SameNL	0	[−12,12]	1.0
Llama	Typed vs. LocObs	+40	[26,54]	2.2×10^{-5}

Table 4: Budget sensitivity on the same 15 games. Cells are raw/typed wins.

Model	$B=2$	$B=4$	$B=6$	$B=8$
Qwen	5/8	4/11	4/11	4/12
Llama	1/5	2/9	2/11	2/11

because it reaches stopping conditions sooner, not because each repair prompt is dramatically smaller.

4.2 Budgets and Sampled Decoding

The budget result is progressive rather than a universal step at call four (Table 4). On 15 paired games, typed-minus-raw gains rise from 20 and 27 points at budget 2 to 47 points for both models at budget 4. Qwen then plateaus near 47–53 points; Llama reaches 60 points at budgets 6 and 8. More calls do not rescue raw retry: its Qwen count is 4/15 at budgets 4–8 and its Llama count is 2/15. The repair interface uses extra compute more effectively.

Sampled Qwen decoding gives the same ordering. Across seeds 3101–3103, RawDiag solves 6, 6, and 5 of 20 games; SameNL solves 14, 13, and 14; and TypedFields solves 14, 16, and 14. Clustered across games, typed versus raw is +45 points (95% interval 23–67; Holm $p = .0053$), while typed versus information-matched prose is +5 points (−3 to 15; $p = .499$). The primary conclusion is not an artifact of greedy repetition.

4.3 When the Validator Cannot Expose the Hidden Failure

The HumanEval lane tests a boundary of the mechanism, not another positive benchmark. Its repair validator executes one visible assertion; the complete official suite remains hidden and is used only for post-hoc success. Qwen's initial candidate passes the public assertion on all 15 tasks under every policy, so no repair feedback is ever issued. Fourteen pass the hidden suite. The remaining hidden failure is therefore invisible to every repair policy, which necessarily produces the identical 14/15 outcome.

Llama creates a few repair opportunities: 4 initial public failures under raw, same-information, and location-only feedback, and 5 under typed feedback. Retrying adds one public-gate success under raw, same-information, and typed feedback, but none under location-only feedback. Only the repaired raw case also passes the hidden suite. Final hidden success is 10/15 for raw, same-information, and location-only feedback and 9/15 for typed feedback.

This result clarifies RQ1's scope. Structured feedback helps only after a validator observes a repair-relevant failure. It cannot reveal behavior that the validator never tests, and passing one visible assertion need not predict hidden correctness. The lane therefore supports a validator-observability boundary; it does not support a claim of general code-repair improvement.

5 IMPLICATIONS

Treat validators as repair APIs. A validator should expose a stable repair locus, the observed value, and actionable expectations when available. For software, these may be a file and line, failing assertion, expected type or behavior, and observed exception. When a test cannot enumerate alternatives, the interface should report what it knows rather than fabricate them.

Content before syntax. JSON is useful for logging, routing, and cross-gate tooling, and our typed condition uses modestly fewer calls. But the same information in prose achieves the same success. Claims that machine-readable feedback inherently improves reasoning would overstate our evidence.

Budget repair, not repetition. Increasing a call cap helps only when later calls receive information that changes the candidate. Raw retry remains flat after four calls, while structured feedback continues to benefit the weaker model through six calls.

6 BOUNDARY CONDITIONS

These boundaries define the claim rather than form a generic checklist. The evidence supports structured feedback for deterministic validators that can expose a repair locus and useful alternatives. It does not establish repository repair or industrial deployment: the primary tasks are 50 generated games, the models are two quantized checkpoints on one serving stack, and sampled decoding uses one model and three seeds. The 15-task HumanEval lane further shows that feedback policy is ineffective when the visible validator misses the hidden failure.

The same-information prose control matches location, expected, and observed values, although wording and delimiters inevitably differ from JSON. Admissible lists are capped at 12 in deterministic environment order; ranking, randomization, and longer lists remain untested. Transcript words are a size proxy, not billed tokens. Real test tools may also be flaky or incomplete, requiring reruns and quarantine. We use Holm correction for the four primary contrasts; budget and code results remain small robustness checks. One transient serving error was quarantined and rerun identically, and only the replacement is analyzed.

7 ARTIFACT AND CONCLUSION

The artifact contains source, deterministic task rules, 880 row-level results, 2,652 per-call traces, model and decoding settings, the quarantined transient row, and a script that regenerates all tables, 10,000-sample bootstrap intervals, exact McNemar tests, Holm corrections, and clustered sampling analysis.

Validator feedback is an agent interface, not incidental logging. Across two models, structured verifier feedback containing

location and alternatives more than doubles TextWorld repair success over raw diagnostics. Information-matched prose performs equally well, so typing is not the causal success mechanism. For agentic development, the practical lesson is precise: expose explicit repair fields first; choose a typed representation for systems reasons, not because this study shows that JSON itself makes an LLM reason better.

REFERENCES

[1] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/2460396f2d0d421885997dd1612ac56b-Abstract-Conference.html

[2] Marc-Alexandre Côté, Àkos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Ruo Yu Tao, Matthew Hausknecht, Layla El Asri, Mahmoud Adada, Wendy Tay, and Adam Trischler. 2018. TextWorld: A Learning Environment for Text-Based Games. *CoRR* abs/1806.11532 (2018). https://arxiv.org/abs/1806.11532

[3] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, et al. 2024. The Llama 3 Herd of Models. *CoRR* abs/2407.21783 (2024). https://arxiv.org/abs/2407.21783

[4] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, An Yang, Rui Men, Fei Huang, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. *CoRR* abs/2409.12186 (2024). https://arxiv.org/abs/2409.12186

[5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html

[6] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html

[7] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html

[8] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X